

Report of NA Project

WangHao 3220103613 *

(Information and Computing Sciences, Class 2201), Zhejiang University

Due time: December 15, 2024

Problem A

This is the N with max-norm of the interpolation error vector.

N	error
6	0.421705
11	0.0205289
21	0.00316894
41	0.000275356
81	0.00001609

Table 1: N and errors

Draw those points in logarithmic graph. It is presented below.

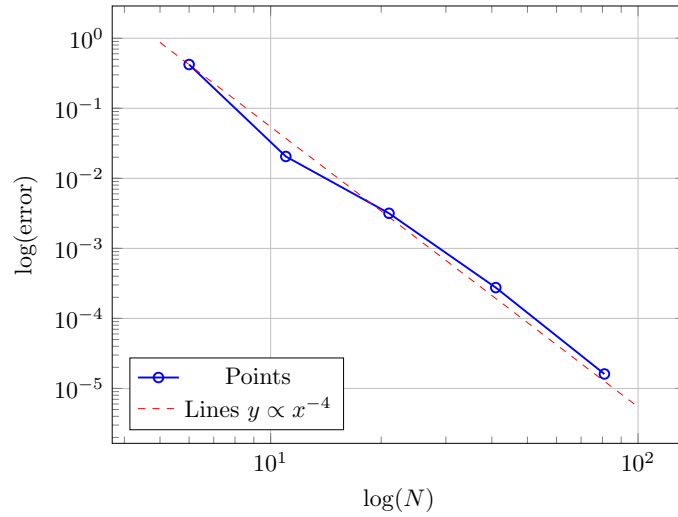


Figure 1: graph of $\log(N)$ with $\log(\text{error})$

We can make a conclusion that the convergence rate is 4, i.e. $\text{error} \propto N^{-4}$.

*Electronic address: 3220103613@zju.edu.cn

The graph of spline and exact function is presented below.

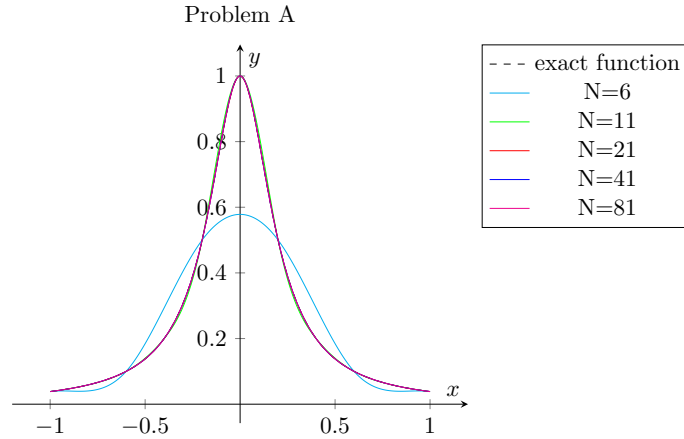


Figure 2: splines against exact function

We can notice that the spline interpolation is free of wide oscillations in Runge phenomenon. It is more precise than polynomial interpolation.

Problem C

Complete boundary condition is used for spline in Theorem 3.57[1] and Thm 3.58 boundary condition is used for spline in Theorem 3.58[2]. The graph of 2 splines and exact function is presented below.

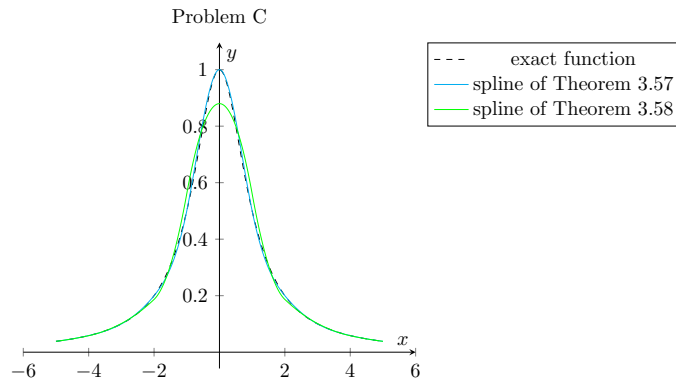


Figure 3: splines against exact function

Problem D

x	$E_S(x)$ of spline in Thm 3.57	$E_S(x)$ of spline in Thm 3.58
-3.5	0.000669568	1.249e-16
-3	3.60822e-16	0.00141838
-0.5	0.0205289	0
0	0	0.120238
0.5	0.0205289	0
3	2.498e-16	0.00141838
3.5	0.000669568	1.38778e-17

Table 2: Comparison of $E_S(x)$ for splines in Thm 3.57 and Thm 3.58.

The reason why some of the errors are close to machine epsilon is that these points are interpolation points of the spline. The spline is constructed by using the condition that they pass through these points. Thus they should be close to machine epsilon.

We can also notice that the spline in Theorem 3.57 performs much better than spline in Theorem 3.58 through graph and table above. This is because many reasons, but I can propose the major one. We all know that interpolation points should fall within interval where the second derivative is huge. By doing this, we can get more precise graphs. And in this case, spline in Theorem 3.57 get the point $(0, 1)$, where the second derivative is hugest, as its interpolation point, while the other don't get that point.

Problem E

i

Apparently we should use periodic boundary conditions, as the curve is a closed curve.

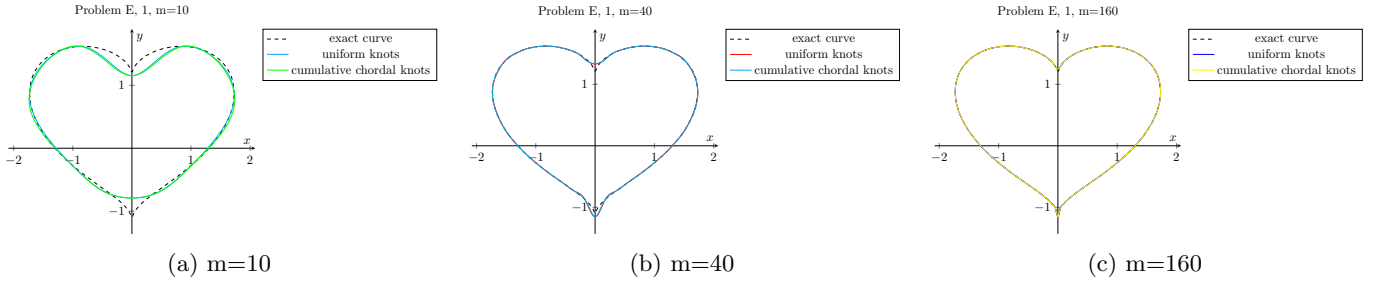


Figure 4: Curve fitting using $spline \in \mathbb{S}_3^2$

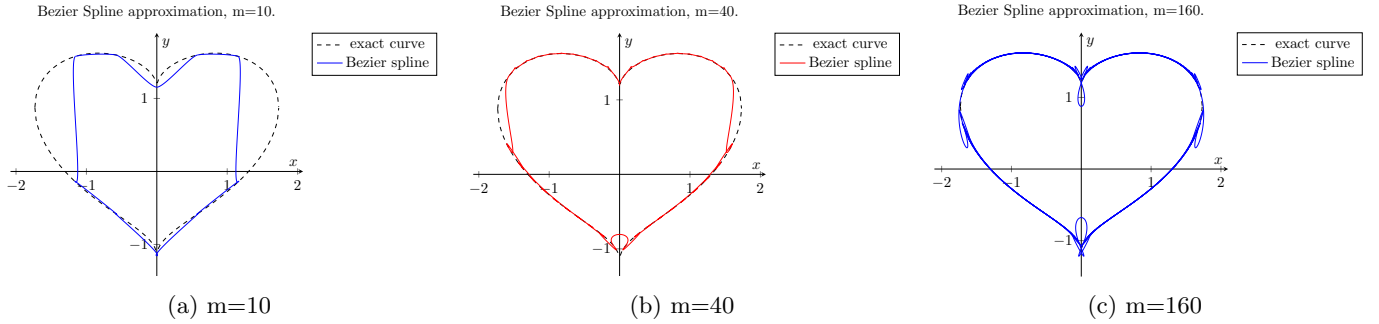


Figure 5: Curve fitting using Bezier Curves

Differences

We can notice that the fitted curve using cubic spline have the tendency to converge to the exact curve when N get large, while cubic Bezier curves seems not.

This is because cubic Bezier curves uses the derivative to generate its control points, which will be far from the exact curve when the derivative get large (actually it converges to infinity in this case). Thus it's not proper to apply it to the curves which have large derivative. While for splines, it generate independent variable list by itself to be uniform or cumulative chordal length, and distribute them in every direction. This means that the spline is independent of derivative. Thus it worked in this situation.

ii

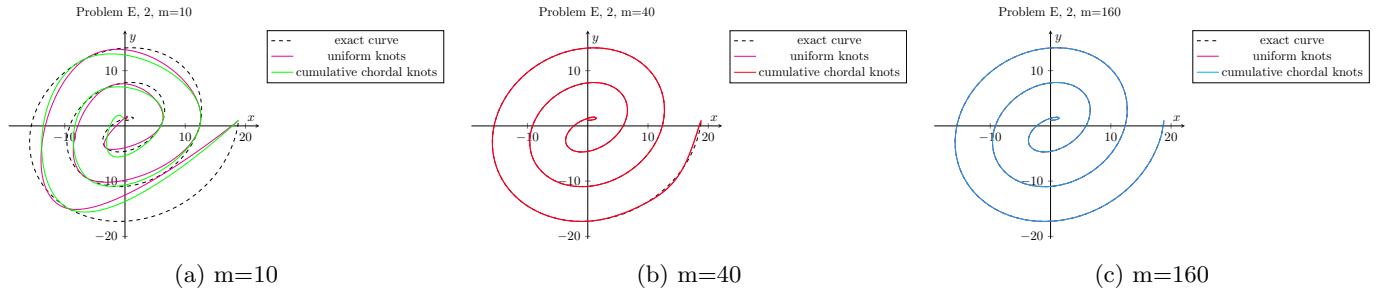


Figure 6: Curve fitting using $spline \in \mathbb{S}_3^2$

iii

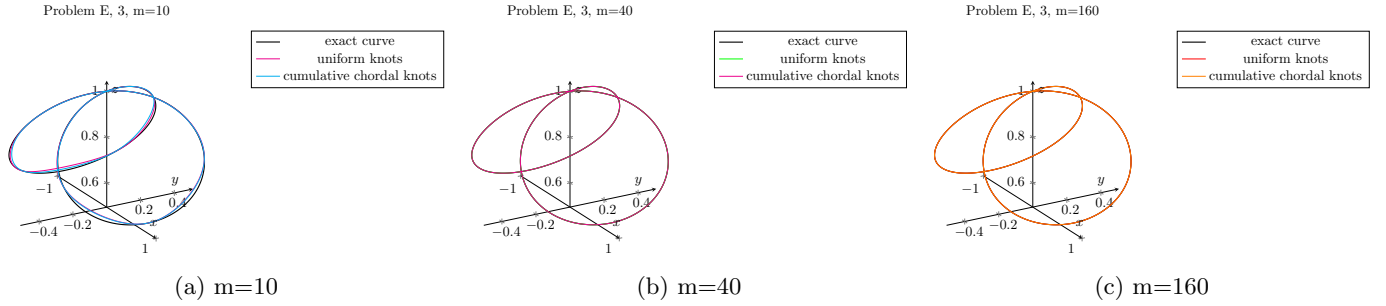


Figure 7: Curve fitting using $spline \in \mathbb{S}_3^2$

Through (ii) and (iii), we can notice that the differences is not significant when m is huge. When m is small, i.e. $m = 10$, there are some differences. In (ii) and (iii), the curvature of the one using cumulative chordal knots is smaller, while the other is bigger, thus having sharper turns.

Problem F

I implement this problem by using class **PiecewisePolynomial**, and create a new class **Table** in F.cpp (as the name of the class is so long, and use name ‘Table’ is Easily confusing, so I use name ‘Table’ and write it in F.cpp). The class **Table** allows arbitrary n and knots. We take $n = 1, 2$ and uniform knots there. Two tables are represented below.

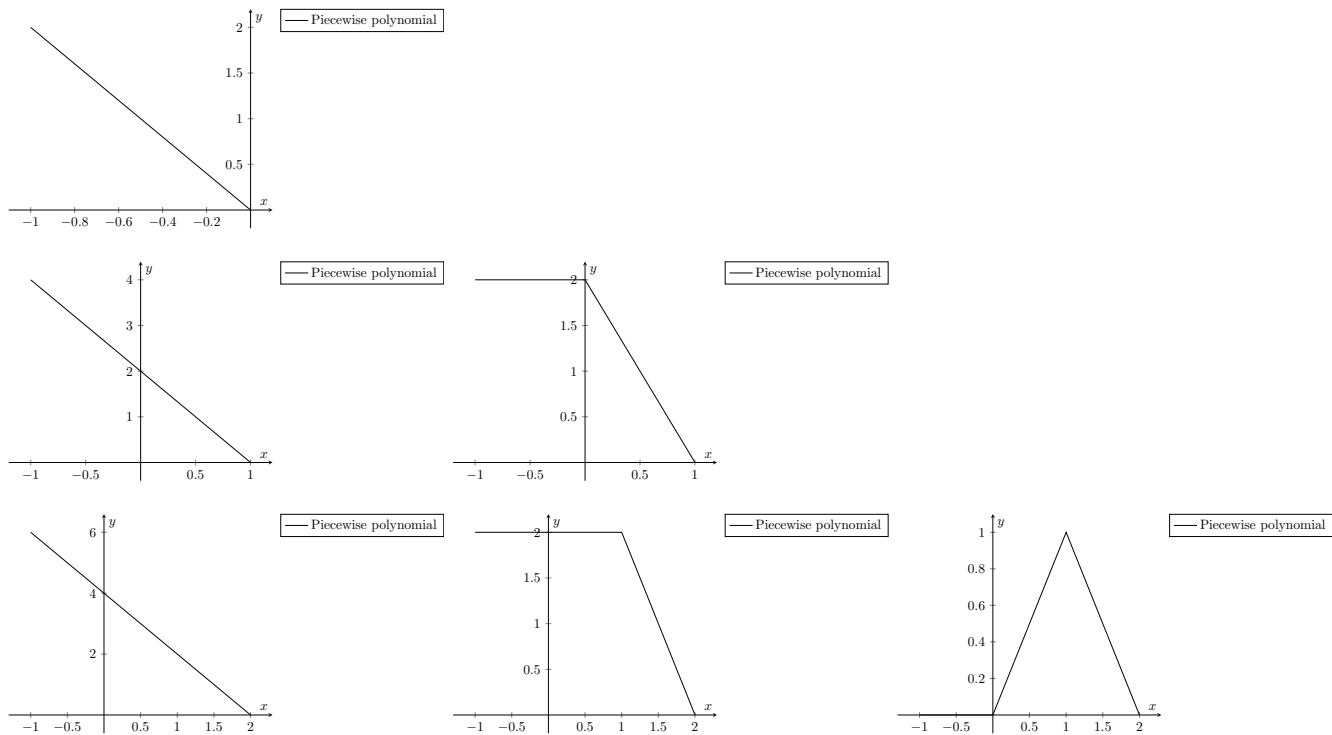


Figure 8: Divided difference table, order=1

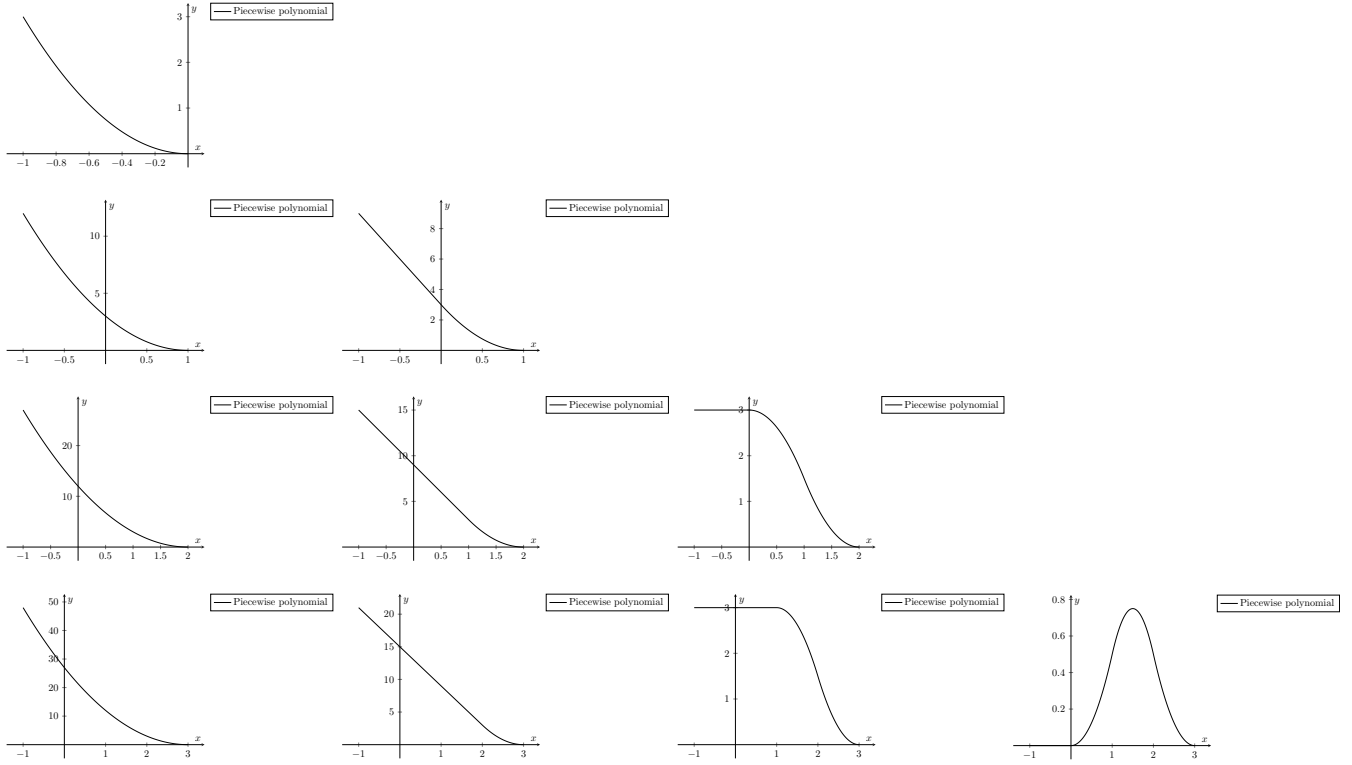
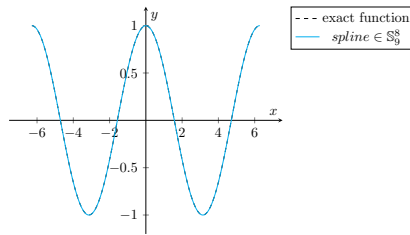


Figure 9: Divided difference table, order=2

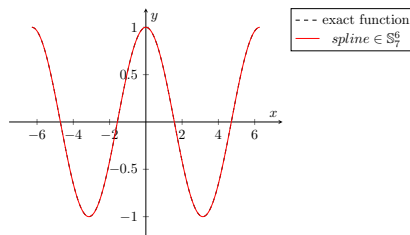
More functions

I do 5 more test to test the splines that have not been tested before, they are done in `moreFunctions.cpp`.

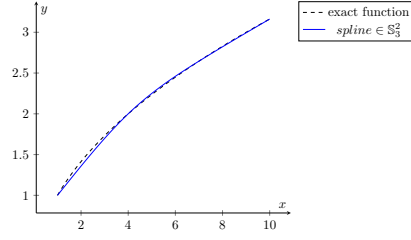
- B-form spline $s \in \mathbb{S}_9^8$. This is to test the B-form spline of arbitrary order. It interpolate $\cos(x)$ in periodic boundary condition, with 5 knots. The graph is shown below.



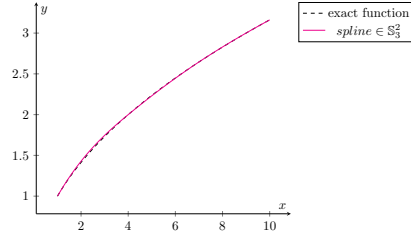
- PP-form spline $s \in \mathbb{S}_7^6$. This is to test the PP-form spline of arbitrary order. It interpolate $\cos(x)$ in periodic boundary condition, with 5 knots. The graph is shown below.



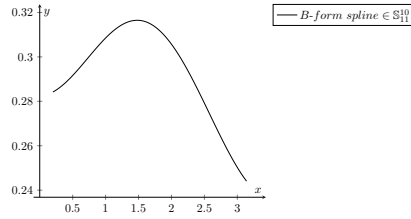
- PP-form spline $s \in \mathbb{S}_3^2$ with natural boundary condition. It interpolate $\sqrt{x}$ with 4 knots. The graph is shown below.



- PP-form spline $s \in \mathbb{S}_3^2$ with complete boundary condition. It interpolate $\sqrt{x}$ with 4 knots. The graph is shown below.



- B-form spline $s \in \mathbb{S}_{11}^{10}$. This is for testing the drawing of spline. The knots are 0.2, 0.6931, 1.14514, 3.14, and the coefficients are 0.1, 0.36, 0.02, 0.42, 0.33, 0.17, 0.38, 0.35, 0.233, 0.1, 0.43, 0.34, 0.51, 0.19. The graph is shown below.



References

- [1] Qinghai Zhang. “Cardinal B-splines, Section 3.2.7”. In: *Handouts NumPDEs*. 2024, p. 29.
- [2] Qinghai Zhang. “Cardinal B-splines, Section 3.2.7”. In: *Handouts NumPDEs*. 2024, p. 29.